Solution: Make The String Great

Problem Statement

Given a string of English lowercase and uppercase letters, make the string "good" by removing two adjacent characters that are the same but in different cases.

Continue to do this until there are no more adjacent characters of the same letter but in different cases. An empty string is also considered "good".

Examples

Example 1

- Input: `"AaBbCcDdEeff"`
- Output: `"ff"`
- Explanation: In the first step, "AaBbCcDdEeff" becomes "BbcCDdEeff" because 'A' and 'a' are the same letter, but one is uppercase and the other is lowercase. Then we remove "Bb", and then "cC", "dD", and "Ee". In the end we are left with "ff" which we can't remove - although both characters are the same but with the same case.

Example 2

- Input: `"abBA"`
- Output: `""`
- Explanation: In the first step, "abBA" becomes "aA" because 'b' and 'B' are the same letter, but one is uppercase and the other is lowercase. Then "aA" becomes "" for the same reason. The final string is empty, which is good.

Example 1

- Input: `"s"`
- Output: `"s"`
- Explanation: The string "s" is already good because it only contains one character.

Solution

To solve this problem, a stack-based approach can be employed. The algorithm iterates through each character in the string. For each character, it checks if the stack is not empty and if the top element of the stack is the opposite case version of the current character (one uppercase and one lowercase of the same letter). If they are, both characters are removed; otherwise, the current character is added to the stack. This process is repeated until the end of the string. The remaining elements in the stack represent the transformed "great" string. The stack ensures that we efficiently manage the removal of adjacent characters and helps to achieve the final string in an optimized way.

Detailed Step-by-Step Algorithm

1. Initialize an empty stack.
2. For each character in the string from left to right:
   - If the stack is not empty and the current character and the top of the stack are the same letter but in different cases, pop the stack.
   - Otherwise, push the current character into the stack.
3. The characters in the stack form the final string. Note that the order of the characters in the stack is the same as their order in the final string.

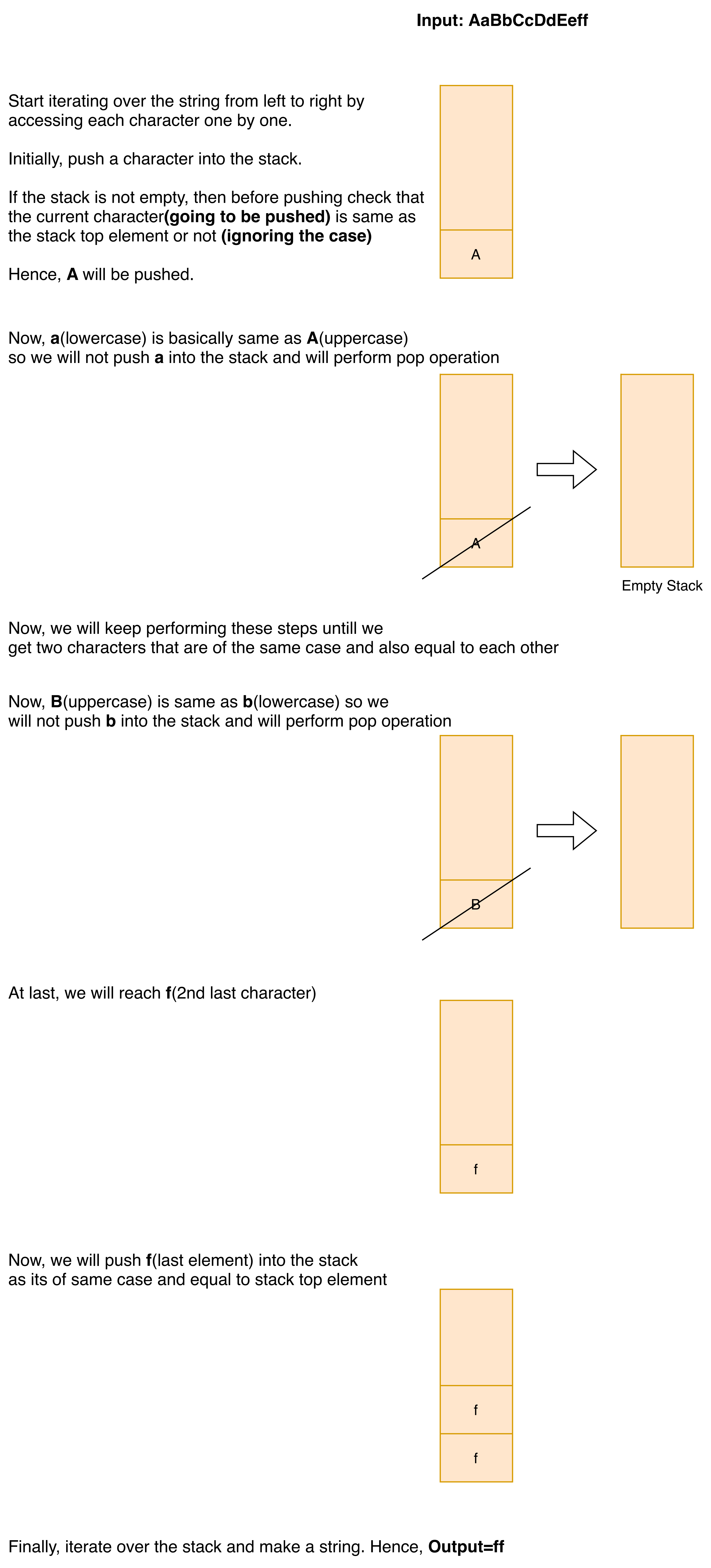
Algorithm walkthrough

Let's consider the input `AaBbCcDdEeff` and observe how the above algorithm works.

1. **Initialize an Empty Stack**: Begin with an empty stack.
2. **Process Each Character**: Go through each character of the string one by one:
   - **Character 'A'**: Stack is empty, so push 'A' onto the stack. Stack: `['A']`.
   - **Character 'a'**: Top of the stack is 'A', which is the uppercase version of 'a'. Pop 'A' from the stack. Stack becomes empty.
   - **Character 'B'**: Stack is empty, so push 'B' onto the stack. Stack: `['B']`.
   - **Character 'b'**: Top of the stack is 'B', which is the uppercase version of 'b'. Pop 'B' from the stack. Stack becomes empty.
   - **Character 'C'**: Stack is empty, so push 'C' onto the stack. Stack: `['C']`.
   - **Character 'c'**: Top of the stack is 'C', which is the uppercase version of 'c'. Pop 'C' from the stack. Stack becomes empty.
   - **Character 'D'**: Stack is empty, so push 'D' onto the stack. Stack: `['D']`.
   - **Character 'd'**: Top of the stack is 'D', which is the uppercase version of 'd'. Pop 'D' from the stack. Stack becomes empty.
   - **Character 'E'**: Stack is empty, so push 'E' onto the stack. Stack: `['E']`.
   - **Character 'e'**: Top of the stack is 'E', which is the uppercase version of 'e'. Pop 'E' from the stack. Stack becomes empty.
   - **Character 'f'**: Stack is empty, so push 'f' onto the stack. Stack: `['f']`.
   - **Character 'f'**: Top of the stack is 'f', but it's not the opposite case version of the current 'f'. Push 'f' onto the stack. Stack: `['f', 'f']`.
3. **Resulting String**: At the end of the iteration, the stack contains `['f', 'f']`. The contents of the stack are then converted back to a string, which gives "ff" as the final output.

So, the transformed "great" string for the input `"AaBbCcDdEeff"` is `"ff"`.

Here is the visual representation of the algorithm:



Input: AaBbCcDdEeff

Start iterating over the string from left to right by accessing each character one by one.

Initially, push a character into the stack.

If the stack is not empty, then before pushing check that the current character**(going to be pushed)** is same as the stack top element or not **(ignoring the case)**

Hence, **A** will be pushed.

Now, **a**(lowercase) is basically same as **A**(uppercase) so we will not push **a** into the stack and will perform pop operation

Now, we will keep performing these steps untill we get two characters that are of the same case and also equal to each other

Now, **B**(uppercase) is same as **b**(lowercase) so we will not push **b** into the stack and will perform pop operation

At last, we will reach **f**(2nd last character)

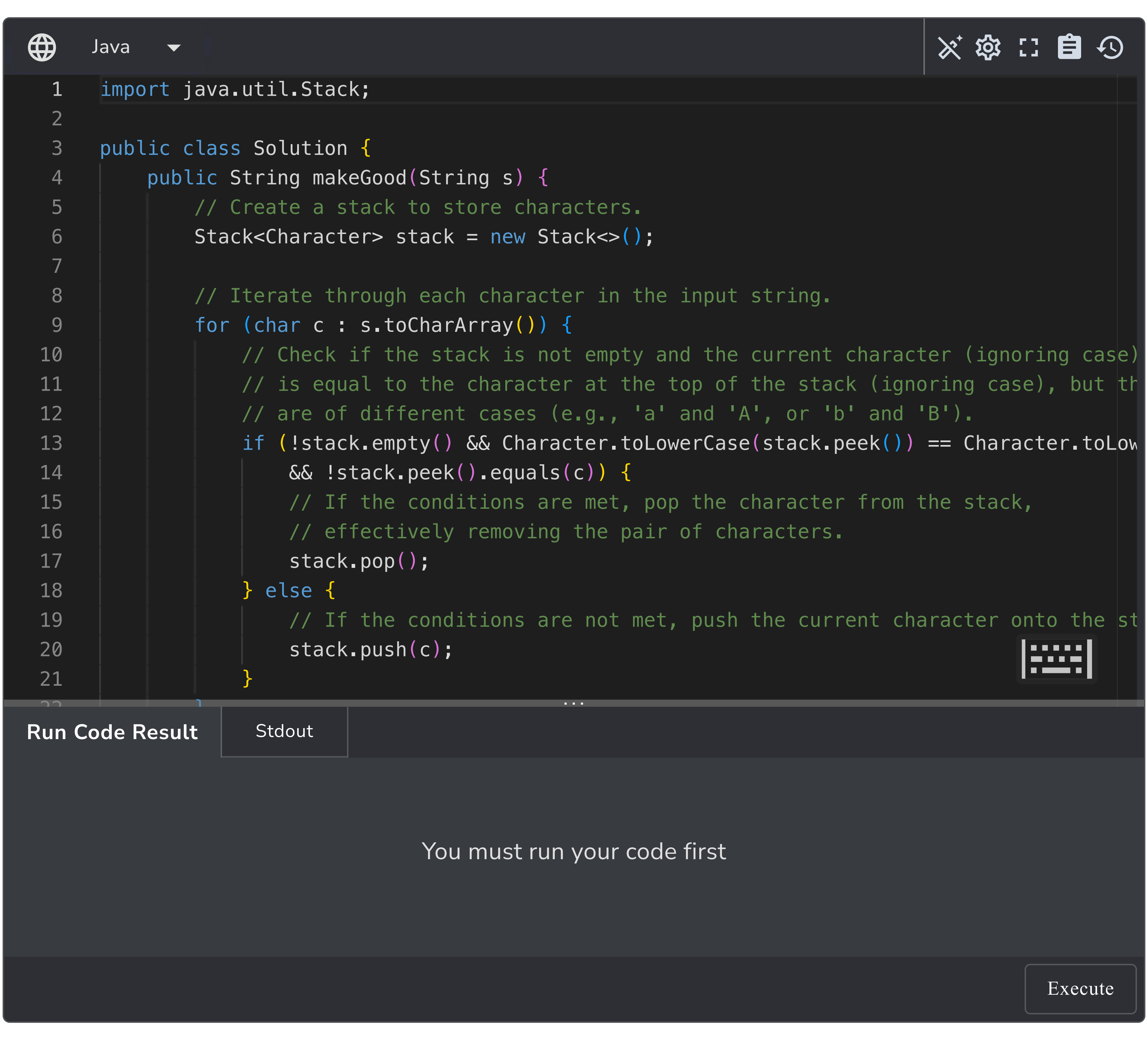
Now, we will push **f**(last element) into the stack as its of same case and equal to stack top element

Finally, iterate over the stack and make a string. Hence, **Output=ff**

Make the string Great

Code

Here is the code for this algorithm:



```java
import java.util.Stack;

public class Solution {
    public String makeGood(String s) {
        // Create a stack to store characters.
        Stack<Character> stack = new Stack<>();

        // Iterate through each character in the input string.
        for (char c : s.toCharArray()) {
            // Check if the stack is not empty and the current character (ignoring case)
            // is equal to the character at the top of the stack (ignoring case), but th
            // are of different cases (e.g., 'a' and 'A', or 'b' and 'B').
            if (!stack.empty() && Character.toLowerCase(stack.peek()) == Character.toLow
                && !stack.peek().equals(c)) {
                // If the conditions are met, pop the character from the stack,
                // effectively removing the pair of characters.
                stack.pop();
            } else {
                // If the conditions are not met, push the current character onto the st
                stack.push(c);
            }
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(n)$, where n is the length of the string. This is because we perform a single pass over the string.

The space complexity of the algorithm is $O(n)$, where n is the length of the string. In the worst case, if the string is already good, the stack will contain all the characters of the string.

← Previous
Problem 9: Make The String Great
Next →
Introduction To Queues
☐ Mark as Completed